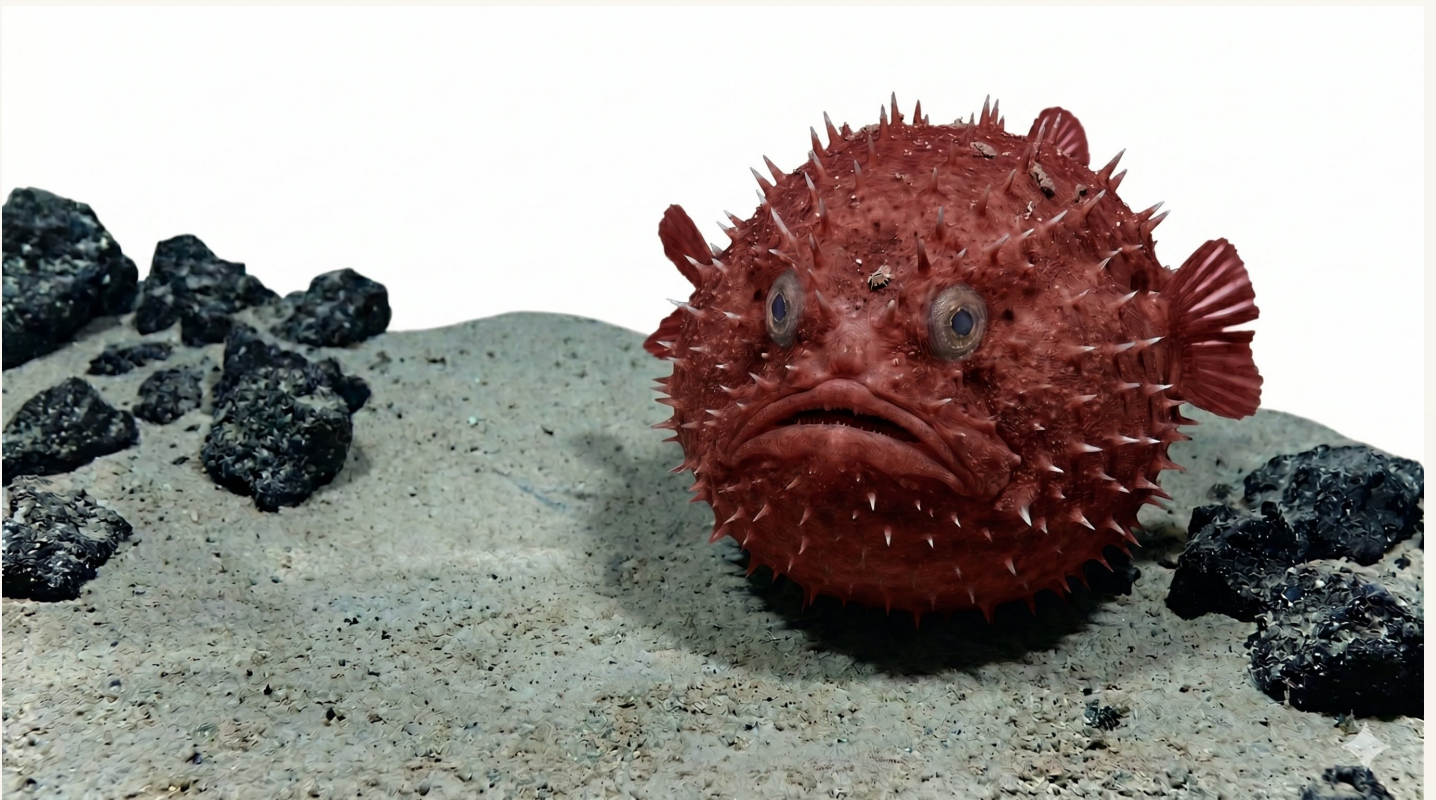


FOUNDATIONS • TRANSFORMERS • SYSTEM DESIGN

Deep Learning for AI Engineers

**The Complete Guide to Neural Networks, Transformers,
Large-Scale AI Systems, and End-to-End Deep Learning
System Design.**

*From a single neuron to a GPT you build from scratch: 17 core chapters,
20 production system case studies, and 6 runnable, gradient-checked
projects, every line of code in the book actually runs.*



Preface

Subscribe → aiengineeringinsider.substack.com/subscribe

Most deep-learning books fail in one of two ways. The academic ones bury the intuition under measure theory and leave you unable to ship anything. The applied ones hand you `model.fit()` and leave you helpless the moment it silently doesn't work. This book is written for the engineer stuck between them: someone who can already code, who wants to *understand* what happens between the data and the prediction, and who will eventually have to debug, optimize, and deploy the thing, not just demo it.

What makes this book different

Three commitments shape every chapter.

First, you build before you borrow. You will implement a perceptron, backpropagation, an autograd engine, a CNN, an RNN, and a GPT *from scratch in NumPy* before you ever call a framework. Only then do PyTorch and JAX appear, as tools whose magic you have already removed. When you later type `loss.backward()`, you will know exactly what it does because you wrote it.

Second, every number is real. There are no idealized outputs in this book. Every console block is captured verbatim from a script that runs, and every hand-derived gradient is checked numerically. While writing the book I found, and fixed, several outputs in early drafts that were simply wrong; the discipline of running everything is the only reliable defense, and it is the discipline this book tries to teach. The complete, tested code lives at <https://github.com/lamhotsiagian/deep-learning>.

Third, the model is only one stage of nine. Real systems are a pipeline, objective, data, preprocessing, splitting, architecture, training, tuning, evaluation, and deployment with monitoring. The hands-on projects walk that full lifecycle, and the twenty production case studies show how Tesla, OpenAI, and others realize each stage at scale. The neural network earns you a seat at the table; the pipeline is how you design the table.

Who this book is for

You should be comfortable writing Python and reading basic linear algebra and calculus (matrix multiplication, partial derivatives, the chain rule). You do *not* need prior machine-learning experience, the book starts at a single neuron. It is aimed at AI and machine-learning engineers, software engineers moving into ML, and anyone preparing for deep-learning or AI system-design interviews, where the questions in each chapter's evaluation are deliberately the ones a strong candidate is asked.

How to use it

Read in order, the chapters compound. Keep a Python REPL open; type the from-scratch code yourself at least once (the NumPy MLP and the character RNN are short enough to hold in your head). Treat the warning boxes as hard-won scar tissue and the chapter evaluations as a self-test before you move on. Section [19.7](#) explains how to set up a virtual environment and run everything.

Lamhot Siagian

AI Engineering Insider

About This Book & AI Engineering Insider

Subscribe → aiengineeringinsider.substack.com/subscribe

About this book

Deep Learning for AI Engineers is a complete, practitioner-grade guide that takes you from a single neuron to a GPT you build, train, and run yourself. It is written for readers who already write code and want the layer underneath the abstractions: the mechanics behind backpropagation, the training tricks that make a network actually learn, the architectures (CNNs, RNNs, and Transformers) that power modern AI, and the system-design thinking behind the products that deploy them at scale.

The book is deliberately end-to-end. Part I builds the foundations from scratch; Part II covers the practitioner’s toolkit (activations, losses, optimizers, initialization, regularization, schedules, evaluation); Part III moves to real frameworks (PyTorch and JAX) and the discipline of debugging; Part IV builds the architecture families, convolutional, recurrent, and the Transformer, from first principles. Two bonus parts then connect theory to practice: twenty production **system-design** case studies (Tesla, ChatGPT, YouTube, AlphaFold, and more), and six **hands-on projects** that walk the full data → train → evaluate → deploy lifecycle, culminating in a Transformer trained from scratch.

★ Key Insight

All code in this book is real, runnable, and tested. Every console output is captured verbatim from a script that actually executes, and every from-scratch gradient is verified numerically. The complete companion repository, per-chapter code, the `coredl` mini-framework, and all six projects, is available at:

<https://github.com/lamhotsiagian/deep-learning>

About AI Engineering Insider

AI Engineering Insider is a technical publication for software engineers and AI practitioners who are tired of content that is long on enthusiasm and short on substance. Every guide is written for readers who already know how to write code and want to go deeper: the mechanics behind the abstractions, the failure modes the documentation doesn’t mention, and the judgment calls that only show up after you’ve debugged the thing at 2 a.m.

Other titles in the AI Engineering Series include:

- *The AI Engineer’s System Design Interview Guide*

- *The AI Engineer's Reinforcement Learning Interview Guide*
- *Transformers & Vision Transformers: 50 Interview Questions*
- *The AI Product Manager's Career Guide*

For new releases, technical deep-dives, and frontier-model analysis, subscribe at <https://aiengineeringinsider.substack.com> or visit <https://aiengineeringinsider.com>.

How to Use This Book

Subscribe → aiengineeringinsider.substack.com/subscribe

This book is built to be read with a pencil, or a Python REPL, in hand. Each chapter follows roughly the same arc: intuition first, then the math, then the mechanics, then where it breaks in practice. Several recurring box types flag different kinds of content so you can scan for what you need:

★ Key Insight

Key Insight marks the one idea in a section that, if you remember nothing else, you should remember this.

! Pitfall / Warning

Pitfall / Warning flags a mistake that is common enough, and costly enough, that it earns its own box.

✓ Best Practice

Best Practice is a concrete, actionable habit worth adopting, not just a thing to know but a thing to do.

From the Field

From the Field is a first-person aside: a war story, a debugging session, or a hard-won lesson that didn't fit neatly into the main text.

Definition

A precise, citable definition of a term, given the first time it matters, separate from the surrounding narrative explanation.

Q0

Interview Q&A boxes pose a question the way an interviewer might ask it.

The answer that follows is written the way a strong candidate would actually say it out loud, not as a textbook proof. Every chapter carries several of these, and they are listed individually in the Table of Contents so you can use this book as interview-prep reference material, not just a cover-to-cover read.

Code listings use a labeled header bar (e.g., `example.py`) and are written to run as-is in a standard Python environment with NumPy and, where relevant, PyTorch or JAX installed. Console output blocks show the actual output you should expect to see, not idealized output, so you can use them to check your own implementation against a known-good run.

Contents

Preface	i
About This Book & AI Engineering Insider	iii
How to Use This Book	v
Part I Foundations	1
1 The Perceptron: Where It All Started	2
1.1 The Biological Inspiration	2
1.2 The Mathematical Perceptron	2
1.3 Architecture Diagram	3
1.4 Geometric Intuition: A Linear Decision Boundary	4
1.5 The Perceptron Learning Rule	5
1.6 The XOR Problem	5
1.7 From Perceptron to Modern Artificial Neuron	6
1.8 Chapter Summary	6
1.9 Chapter Evaluation	6
1.10 Answers	8
2 Multi-Layer Networks & Forward Pass	11
2.1 Why Depth: The Universal Approximation Theorem	11
2.2 Layers, Weights, and Biases as Matrices	11
2.3 The Forward Pass	12
2.4 Architecture Diagram	13
2.5 Width vs. Depth	15
2.6 Computational Graphs	15
2.7 Chapter Summary	16
2.8 Chapter Evaluation	16
2.9 Answers	18
3 Backpropagation from Scratch	21
3.1 The Credit Assignment Problem	21
3.2 Chain Rule Refresher	21
3.3 Backprop Through a Single Neuron	22
3.4 Backprop Through a Layer (Vectorized)	22
3.5 Backprop Through Multiple Layers	23
3.6 Architecture Diagram	24
3.7 Computational Graphs and Reverse-Mode Autodiff	24
3.8 Common Derivation Mistakes	25
3.9 Vanishing and Exploding Gradients: First Encounter	25

3.10	Chapter Summary	26
3.11	Chapter Evaluation	26
3.12	Answers	27
4	Build Your Own Mini Framework	30
4.1	Why Build a Toy Framework	30
4.2	Designing a Value Class	30
4.3	Implementing Forward Operations	31
4.4	The Backward Pass: Topological Sort	33
4.5	Architecture Diagram	33
4.6	Building a Tiny MLP	34
4.7	Training It End to End	35
4.8	Gradient Checking Your Engine	36
4.9	Chapter Summary	37
4.10	Chapter Evaluation	37
4.11	Answers	39
Part II	Making Networks Learn Well	42
5	Activation Functions: ReLU, Sigmoid, GELU & Why	43
5.1	Sigmoid and Tanh: The Original Choices	43
5.2	ReLU: Simplicity, Sparsity, and Dying Units	44
5.3	Leaky ReLU and Parametric ReLU	45
5.4	GELU and Swish/SiLU	45
5.5	Architecture Diagram	46
5.6	Softmax for Output Layers	46
5.7	Choosing an Activation by Layer Position and Task	47
5.8	Chapter Summary	48
5.9	Chapter Evaluation	48
5.10	Answers	49
6	Loss Functions: MSE, Cross-Entropy, Contrastive	52
6.1	Regression Losses	52
6.2	Classification Losses	53
6.3	Why Cross-Entropy Pairs with Softmax	53
6.4	Contrastive Loss	54
6.5	Architecture Diagram	55
6.6	Choosing a Loss Function for Your Task	56
6.7	Diagnosing a Loss That Won't Go Down	56
6.8	Chapter Summary	57
6.9	Chapter Evaluation	57
6.10	Answers	58
7	Optimizers: SGD, Momentum, Adam, AdamW	61
7.1	Batch, Stochastic, and Mini-Batch Gradient Descent	61
7.2	Momentum	61
7.3	Adagrad and RMSProp: Adaptive Learning Rates	62
7.4	Adam: Combining Momentum and Adaptive Rates	62

7.5	AdamW: Decoupling Weight Decay	64
7.6	Architecture Diagram	64
7.7	Practical Optimizer Selection	65
7.8	Chapter Summary	65
7.9	Chapter Evaluation	66
7.10	Answers	67
8	Weight Initialization & Training Stability	70
8.1	Why Naive Initialization Fails	70
8.2	Signal Propagation: Keeping Variance Stable	71
8.3	Xavier/Glorot Initialization	71
8.4	He Initialization	71
8.5	Architecture Diagram	72
8.6	Diagnosing Bad Initialization in Practice	73
8.7	Chapter Summary	74
8.8	Chapter Evaluation	75
8.9	Answers	76
9	Regularization: Dropout, Weight Decay, BatchNorm	79
9.1	Overfitting vs. Underfitting, Revisited	79
9.2	Architecture Diagram	79
9.3	L1 and L2 Weight Regularization	80
9.4	Dropout	80
9.5	Batch Normalization	81
9.6	Layer Normalization vs. Batch Normalization	82
9.7	Data Augmentation	82
9.8	Early Stopping	83
9.9	Chapter Summary	83
9.10	Chapter Evaluation	84
9.11	Answers	85
10	Learning Rate Schedules & Warmup	88
10.1	Why a Fixed Learning Rate Is Rarely Optimal	88
10.2	Step Decay and Exponential Decay	88
10.3	Cosine Annealing	88
10.4	Warmup	89
10.5	Cyclical Learning Rates and the LR Range Test	90
10.6	Architecture Diagram	91
10.7	Practical Scheduling Recipes	92
10.8	Chapter Summary	92
10.9	Chapter Evaluation	92
10.10	Answers	94
11	Evaluation & Diagnosis	96
11.1	Train, Validation, and Test Splits	96
11.2	Accuracy, Precision, Recall, F1	96
11.3	Confusion Matrices	97
11.4	ROC Curves and AUC	98

11.5	k-Fold Cross-Validation	98
11.6	Reading Loss Curves	99
11.7	Architecture Diagram	99
11.8	Hyperparameter Tuning Workflow	100
11.9	Chapter Summary	100
11.10	Chapter Evaluation	101
11.11	Answers	102
Part III Real Frameworks		105
12	Introduction to PyTorch	106
12.1	Tensors: PyTorch's Core Data Structure	106
12.2	Autograd: Comparing to Your Chapter 4 Engine	106
12.3	nn.Module: Building Networks Declaratively	107
12.4	Optimizers and the Training Loop	108
12.5	DataLoader and Dataset	109
12.6	GPU Placement	110
12.7	Saving, Loading, and Checkpointing	111
12.8	A Minimal, Correct Training Loop Template	111
12.9	Architecture Diagram	112
12.10	Chapter Summary	112
12.11	Chapter Evaluation	113
12.12	Answers	114
13	Introduction to JAX	117
13.1	JAX's Philosophy: Pure Functions, No Hidden State	117
13.2	grad: Automatic Differentiation as a Function Transformation	117
13.3	jit: Just-In-Time Compilation	119
13.4	vmap: Automatic Vectorization	120
13.5	PyTrees	120
13.6	A Minimal Training Loop in JAX	121
13.7	Architecture Diagram	122
13.8	JAX vs. PyTorch: When Each Makes Sense	123
13.9	Chapter Summary	123
13.10	Chapter Evaluation	123
13.11	Answers	125
14	Debugging Neural Networks	128
14.1	The Debugging Mindset: Isolate Before You Tune	128
14.2	Shape Errors and How to Localize Them Fast	128
14.3	NaN/Inf Losses: Common Causes and Fixes	129
14.4	Loss Not Decreasing: A Decision Tree	130
14.5	Loss Decreasing but Model Not Generalizing	131
14.6	Gradient Checking and Gradient Flow Visualization	131
14.7	Overfitting a Single Batch as a Sanity Test	132
14.8	Reproducibility: Seeding and Determinism	133
14.9	Architecture Diagram	133

14.10 Chapter Summary	133
14.11 Chapter Evaluation	134
14.12 Answers	135
Part IV Architectures	138
15 Convolutional Neural Networks	139
15.1 Why Fully Connected Layers Fail on Images	139
15.2 The Convolution Operation	139
15.3 Stride, Padding, and Output Size	141
15.4 Pooling: Max and Average	141
15.5 Receptive Fields	142
15.6 The Canonical CNN Pattern	143
15.7 Architecture Diagram	144
15.8 A Brief Tour of Milestone Architectures	144
15.9 Transfer Learning	145
15.10 Chapter Summary	145
15.11 Chapter Evaluation	145
15.12 Answers	147
16 Recurrent Neural Networks, LSTM & GRU	150
16.1 Sequential Data and the Limits of Feedforward Networks	150
16.2 The Vanilla RNN: Hidden State as Memory	150
16.3 Backpropagation Through Time	151
16.4 Vanishing Gradients in RNNs, Revisited	151
16.5 LSTM: Forget, Input, and Output Gates	152
16.6 Architecture Diagram	153
16.7 GRU: A Simplified Gating Alternative	153
16.8 LSTM vs. GRU: When Each Is Preferable	154
16.9 Where RNNs Still Matter Today	154
16.10 Chapter Summary	155
16.11 Chapter Evaluation	155
16.12 Answers	156
17 Transformers & Attention	159
17.1 Transformer Foundations	159
17.2 The Self-Attention Mechanism	164
17.3 Multi-Head Attention	168
17.4 Encoder–Decoder Architecture	172
17.5 Vision Transformers (ViT)	175
17.6 CNN vs. ViT, Hybrids, and Modern Techniques	177
17.7 Quick Reference	181
Part V Bonus: Systems at Scale	182
18 Bonus: 20 Deep Learning Systems in Production	183
18.1 The Twenty Systems at a Glance	183

18.2	Case 1: Tesla Autopilot & Full Self-Driving	185
18.3	Case 2: OpenAI ChatGPT	186
18.4	Case 3: Google Search Ranking	187
18.5	Case 4: YouTube Recommendations	188
18.6	Case 5: Netflix Recommendations	189
18.7	Case 6: Meta Feed & Ads Ranking (DLRM)	189
18.8	Case 7: Spotify Music Recommendation	190
18.9	Case 8: Uber DeepETA	191
18.10	Case 9: Amazon (Recommendations & Alexa)	192
18.11	Case 10: TikTok / ByteDance Monolith	193
18.12	Case 11: DeepMind AlphaFold 2	193
18.13	Case 12: Google Translate	194
18.14	Case 13: Pinterest PinSage	195
18.15	Case 14: Waymo Autonomous Driving	196
18.16	Case 15: Stable Diffusion / Latent Diffusion	196
18.17	Case 16: GitHub Copilot	197
18.18	Case 17: Stripe Radar (Fraud Detection)	198
18.19	Case 18: LinkedIn Feed & Connections	199
18.20	Case 19: DoorDash / Instacart Logistics	199
18.21	Case 20: NVIDIA DLSS	200
18.22	The Twenty Systems as End-to-End Pipelines	201
18.23	Cross-Cutting Lessons	202
Part VI Bonus: Build It Yourself		203
19 Bonus: Six Hands-On Projects, Step by Step		204
19.1	Project 1: A Multi-Layer Perceptron in Pure NumPy	205
19.2	Project 2: A Custom ResNet on CIFAR-10	208
19.3	Project 3: A Character-Level RNN in Pure NumPy	211
19.4	Project 4: Seq2Seq with Bahdanau Attention	214
19.5	Project 5: A VAE and a Diffusion Model	216
19.6	Project 6: A GPT-Style Transformer From Scratch	219
19.7	Where to Go Next	222
Running the Code		223
19.8	Repository Layout	223
19.9	Step 1: Create a Virtual Environment	223
19.10	Step 2: Install the Requirements	223
19.11	Step 3: Run It	224
Appendix A: Notation Reference		226
Common Abbreviations		227
Appendix B: Glossary		228
Appendix C: Recommended Next Steps		231
If You Want to Go Deeper on Architecture		231
If You Want to Go Deeper on Training at Scale		231
If You Want to Go Deeper on LLMs Specifically		231

If You Want Interview-Ready Depth	232
A Closing Note	232
References	233

PART I

Foundations

Every architecture, every optimizer, every training trick in modern deep learning rests on four ideas: a neuron, a forward pass, backpropagation, and the discipline of building it yourself before you trust a framework to do it for you. This part is slow on purpose. If Chapters 1 through 4 are solid, everything else in this book is just variation on a theme.

The Perceptron: Where It All Started

Subscribe → aiengineeringinsider.substack.com/subscribe

The original 1958 idea, why it failed, and why it still matters.

Every neural network you will ever train, from a two-layer toy model to a hundred-billion-parameter transformer, is built out of the same atomic unit. That unit was first formalized in 1958 by Frank Rosenblatt as the **perceptron**. It is worth spending real time here, not because the perceptron itself is useful (it isn't, on its own), but because every confusion you will later have about activations, decision boundaries, and why depth matters traces back to understanding this one simple model precisely.

1.1 The Biological Inspiration

The perceptron was loosely inspired by the biological neuron: dendrites receive signals, the cell body aggregates them, and if the aggregate signal is strong enough, the neuron fires an output along its axon. Rosenblatt's insight was to formalize “aggregate, then decide” as a mathematical operation that could, in principle, be learned from data rather than hand-programmed.

! Pitfall / Warning

The biological analogy is useful for building intuition once, but stop using it as soon as the math is in front of you. Real biological neurons are vastly more complex (dendritic computation, spike timing, plasticity rules that bear no resemblance to gradient descent), and over-relying on the analogy tends to produce sloppy intuitions about what artificial neurons actually compute.

1.2 The Mathematical Perceptron

Strip away the biology and the perceptron is a precise, small piece of math. Given an input vector $\mathbf{x} \in \mathbb{R}^n$, a weight vector $\mathbf{w} \in \mathbb{R}^n$, and a scalar bias b , the perceptron computes:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b$$

and then applies a step function to produce a binary output:

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Perceptron

A perceptron is a linear classifier: it computes a weighted sum of its inputs plus a bias, then applies a hard threshold (the step function) to produce a binary decision. The weights and bias are the only learnable parameters.

That's the entire forward computation. No hidden layers, no nonlinearity beyond the final threshold, just a weighted sum and a yes/no decision.

1.2.1 A Worked Example

Suppose you want a perceptron to decide whether to approve a small loan based on two features: credit score (normalized to $[0, 1]$) and existing debt (normalized to $[0, 1]$, higher is worse). A plausible (untrained, hand-picked for illustration) set of weights might be $w_1 = 3$ (credit score, positive influence), $w_2 = -2$ (debt, negative influence), and $b = -1$.

perceptron_example.py

```
def perceptron(x1, x2, w1=3.0, w2=-2.0, b=-1.0):
    z = w1 * x1 + w2 * x2 + b
    return 1 if z >= 0 else 0

# High credit score, low debt -> approve
print(perceptron(0.9, 0.1))    # z = 2.7 - 0.2 - 1 = 1.5 -> 1

# Low credit score, high debt -> deny
print(perceptron(0.2, 0.8))    # z = 0.6 - 1.6 - 1 = -2.0 -> 0
```

Output

1
0

1.3 Architecture Diagram

Figure 1.1 shows the full perceptron forward pass as a data-flow diagram: each input feature x_i is scaled by its weight w_i , the scaled signals (plus the bias b) are summed into the pre-activation z , and a hard step function maps z to a binary decision. Read it left to right; this is the exact computation every later neuron in this book generalizes.

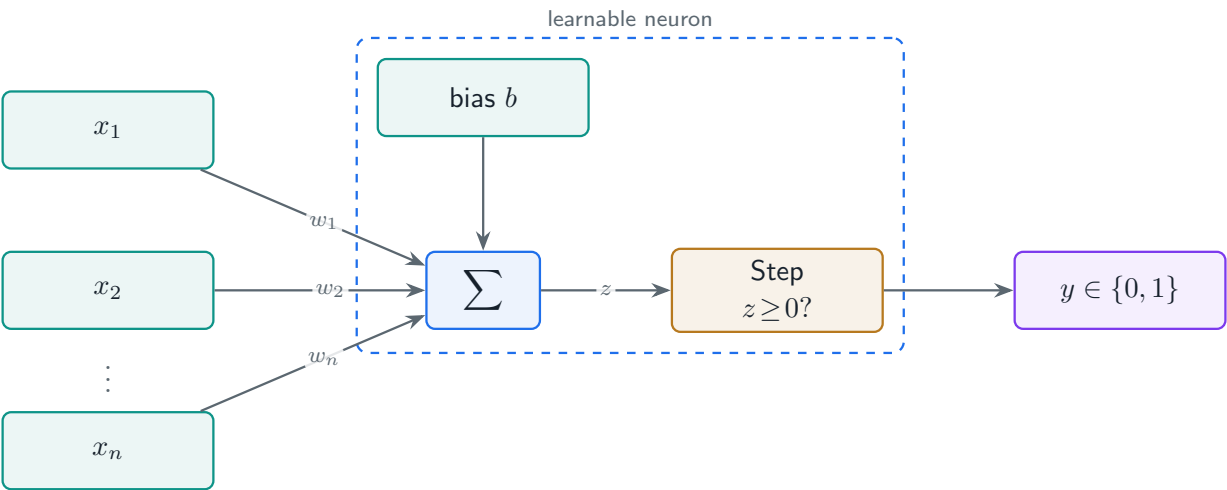


Figure 1.1: The perceptron forward pass: weighted sum followed by a hard threshold. The dashed box is the single learnable “neuron”; everything inside it is reused, unchanged, by every modern artificial neuron.

Table 1.1 contrasts the perceptron with the two models it is most often confused with. The structural core (a weighted sum) is identical; the output map and training rule are what differ, and those differences are exactly why gradient-based deep learning exists.

Table 1.1: Perceptron versus logistic regression versus the modern artificial neuron.

Property	Perceptron (1958)	Logistic Regression	Modern Neuron
Output map	Hard step $\{0, 1\}$	Sigmoid $\rightarrow (0, 1)$	Smooth activation (ReLU, GELU, ...)
Output meaning	Class label	Calibrated probability	Pre/post-activation feature
Differentiable?	No (zero a.e.)	Yes	Yes
Trained by	Perceptron rule	Gradient descent on cross-entropy	Backprop + SGD/Adam
Convergence	Guaranteed iff linearly separable	Convex, global optimum	Non-convex (in a network)
Composable into layers?	Not usefully	Rarely	Yes — the entire point

1.4 Geometric Intuition: A Linear Decision Boundary

The equation $\mathbf{w}^\top \mathbf{x} + b = 0$ defines a hyperplane in input space. In two dimensions, that hyperplane is just a line. Everything on one side of the line gets classified as 1; everything on the other side gets classified as 0. The weight vector $\mathbf{w}$ is normal (perpendicular) to that line, and the bias b shifts the line away from the origin.

★ Key Insight

A perceptron can only ever carve up its input space with a single straight line (or, in higher dimensions, a single flat hyperplane). If your data isn’t **linearly separable**, no setting of $\mathbf{w}$ and b will ever classify it perfectly. This single fact is the entire reason multi-layer networks exist.

1.5 The Perceptron Learning Rule

Rosenblatt also gave the perceptron a learning algorithm, and it predates backpropagation by decades. For each misclassified training example $(\mathbf{x}, y_{\text{true}})$, update the weights:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y_{\text{true}} - y_{\text{pred}}) \mathbf{x} \quad b \leftarrow b + \eta(y_{\text{true}} - y_{\text{pred}})$$

where η is the learning rate. If the prediction was correct, the update is zero and nothing changes. If the perceptron predicted 0 but the true label was 1, the weights nudge toward the input. If it predicted 1 but the true label was 0, the weights nudge away.

✓ Best Practice

Implement the perceptron learning rule from scratch at least once, on a tiny 2D linearly separable dataset, and plot the decision boundary after each epoch. Watching the line rotate and shift into place is the single fastest way to build real intuition for what “learning” means in a neural network, before gradients and loss surfaces enter the picture.

The **Perceptron Convergence Theorem** guarantees that if the data is linearly separable, this algorithm will find a separating hyperplane in a finite number of steps. It says nothing about what happens if the data is not linearly separable, and that gap turned out to be the whole problem.

1.6 The XOR Problem

In 1969, Marvin Minsky and Seymour Papert published *Perceptrons*, a rigorous analysis of what single-layer perceptrons could and could not compute. Their most famous result: a single perceptron cannot learn the XOR function.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Plot these four points. The two points labeled 1 sit on opposite corners from each other, and so do the two points labeled 0. No single straight line can separate the 1s from the 0s. XOR is not linearly separable, so by the geometric argument above, no perceptron, no matter how it’s trained, can solve it.

! Pitfall / Warning

This result is widely (and somewhat unfairly) credited with triggering the first “AI winter.” Funding for neural network research dried up for roughly a decade, in part because Minsky and Papert’s result was read more broadly than it actually applied, it’s a statement about *single-layer* perceptrons, not neural networks in general. The fix (stacking layers) was already conceptually available; what was missing was an efficient way to train multi-layer

networks, which is exactly the gap backpropagation later filled.

1.7 From Perceptron to Modern Artificial Neuron

The modern artificial neuron used in deep learning keeps the weighted-sum structure but changes two things. First, the hard step function is replaced with a smooth, differentiable activation function (Chapter 5), because a discontinuous step function has a derivative of zero almost everywhere, which makes gradient-based learning impossible. Second, neurons are stacked into layers, and layers are stacked into networks, which is exactly the escape hatch from the XOR problem (Chapter 2).

Perceptron vs. Modern Artificial Neuron

A **perceptron** uses a hard step function and the historical perceptron learning rule. A **modern artificial neuron** uses the same weighted-sum computation but pairs it with a smooth, differentiable activation function and is trained with gradient descent and backpropagation, not the original perceptron rule. The terms are often used loosely as synonyms in casual conversation, but the distinction matters the moment you start asking “how is this trained?”

1.8 Chapter Summary

- A perceptron computes a weighted sum of its inputs plus a bias, then applies a hard threshold.
- Geometrically, a perceptron defines a single linear decision boundary, a hyperplane.
- The perceptron learning rule provably converges, but only on linearly separable data.
- XOR is the canonical example of a non-linearly-separable function, and it cannot be learned by a single perceptron.
- Modern neurons replace the step function with a smooth activation function and are trained with gradient descent, not the original rule, setting up everything in Chapter 2 onward.

1.9 Chapter Evaluation

Ten questions spanning the whole chapter: conceptual foundations, a production-style design call, and a debugging scenario. Try each with only the hint first; full worked answers follow in section 1.10.

Exercise 1.1

Why can a single perceptron never learn the XOR function, and what is the smallest change that fixes it?

Hint. Think about what shape a single decision boundary can take, then plot the four XOR points.

Exercise 1.2

Structurally a perceptron and logistic regression both compute $\mathbf{w}^\top \mathbf{x} + b$. Name the two things that differ, and why each matters in practice.

Hint. One difference is in the output map; the other is in the training signal. Both trace back to differentiability.

Exercise 1.3

The perceptron learning rule is $\mathbf{w} \leftarrow \mathbf{w} + \eta(y_{\text{true}} - y_{\text{pred}})\mathbf{x}$. Explain geometrically what each update does to the decision boundary.

Hint. $\mathbf{w}$ is normal to the boundary. A misclassified positive example pushes $\mathbf{w}$ which way?

Exercise 1.4

The Perceptron Convergence Theorem guarantees a solution. State its precondition, and explain why the guarantee is nearly useless in practice.

Hint. Real data is almost never perfectly separable. What does the theorem promise when it isn't?

Exercise 1.5

Why is the step function fatal for gradient-based learning, and what replaced it?

Hint. Differentiate the step function. What is its slope almost everywhere, and what does that do to a gradient update?

Exercise 1.6

(Production) You are asked to ship a binary fraud classifier. A junior engineer proposes “a perceptron, it’s the simplest neural net.” Give two concrete technical reasons to reject that and one to keep the linear model anyway.

Hint. Think calibration, thresholds, and the cost of a non-differentiable output in a monitored production system.

Exercise 1.7

(Debugging) You implement the perceptron rule and the weights oscillate forever without converging. List three distinct causes and how you’d confirm each.

Hint. One cause is in the data, one in the learning rate, one in your stopping condition.

Exercise 1.8

Implement the perceptron learning rule from scratch in NumPy and train it on a 2D linearly separable set. What single line decides convergence?

Hint. The loop should stop when no example is misclassified in a full pass.

Exercise 1.9

Explain how stacking perceptrons solves XOR at the level of decision regions, not just “add a hidden layer.”

Hint. Each hidden unit draws one line. What does the output unit do with two half-planes?

Exercise 1.10

(Technical) The bias b is often folded into $\mathbf{w}$ by appending a constant-1 feature. Show the algebra and state one practical reason frameworks keep bias separate anyway.

Hint. Augment $\mathbf{x} \rightarrow [\mathbf{x}, 1]$ and $\mathbf{w} \rightarrow [\mathbf{w}, b]$. Then think about weight decay.

1.10 Answers

Answer 1.1

Why can a single perceptron never learn XOR, and what is the smallest fix?

XOR is not **linearly separable**. A perceptron's rule $\mathbf{w}^\top \mathbf{x} + b \geq 0$ defines a single hyperplane (a line in 2D) splitting input space into two half-planes. Plot the four points: the positive class $\{(0, 1), (1, 0)\}$ sits on opposite corners from the negative class $\{(0, 0), (1, 1)\}$, so no single line can separate them. The smallest fix is one hidden layer with two units: each hidden unit learns a different line (e.g. " x_1 OR x_2 " and "NOT (x_1 AND x_2)"), and the output unit ANDs those two half-planes, carving out the XOR region. This is precisely the motivation for chapter 2.

Answer 1.2

What differs between a perceptron and logistic regression?

Two things. (1) **Output map:** the perceptron applies a hard step and emits a class label; logistic regression applies a sigmoid and emits a calibrated probability in $(0, 1)$. Calibration matters whenever a downstream system thresholds on confidence (ranking, cost-sensitive decisions). (2) **Training signal:** the perceptron updates only on misclassification by a fixed amount; logistic regression minimizes cross-entropy by gradient descent. Because the sigmoid is smooth, its loss is differentiable everywhere, which is what lets the same machinery scale to every deep network in this book. The perceptron's hard threshold has zero gradient almost everywhere, so it cannot be trained that way.

Answer 1.3

Geometric meaning of each perceptron update.

$\mathbf{w}$ is normal to the decision boundary, so changing $\mathbf{w}$ rotates the boundary and changing b translates it. For a false negative ($y_{\text{true}} = 1, y_{\text{pred}} = 0$) the update adds $\eta \mathbf{x}$, rotating $\mathbf{w}$ toward the misclassified point so its projection $\mathbf{w}^\top \mathbf{x}$ grows and eventually crosses zero. A false positive subtracts $\eta \mathbf{x}$, rotating away. A correct prediction yields a zero update. Each step reduces the margin violation of the offending point, though possibly at the expense of others, which is why it only provably terminates on separable data.

Answer 1.4*Precondition and limitation of the Convergence Theorem.*

Precondition: the training set is *linearly separable*. Under that assumption the rule finds a separating hyperplane in a finite number of updates (bounded by $(R/\gamma)^2$ for radius R and margin γ). It is nearly useless in practice because (a) real data is almost never perfectly separable, and on non-separable data the rule *never* converges, it cycles forever; (b) it says nothing about the *quality* of the boundary (no max-margin guarantee like an SVM); and (c) the bound depends on $1/\gamma^2$, which can be astronomically large for nearly-overlapping classes.

Answer 1.5*Why the step function breaks gradient learning.*

The step function is flat everywhere except at $z = 0$, where it is discontinuous. Its derivative is therefore 0 almost everywhere (and undefined at the jump). Backpropagation multiplies by this local derivative, so every gradient that flows through a step unit is annihilated, the network receives no learning signal. It was replaced by smooth activations (sigmoid, tanh, and later ReLU/GELU in chapter 5) whose nonzero derivatives let gradient information propagate.

Answer 1.6*Production: perceptron for a fraud classifier?*

Reject because: (1) the hard $\{0,1\}$ output gives no score to threshold, fraud systems need a tunable probability so you can trade precision against recall and feed a calibrated risk into downstream rules; (2) the non-differentiable output blocks gradient-based training and modern tooling (early stopping on a smooth validation loss, probability calibration, monitoring of score drift). **Keep a linear model anyway because:** if the signal is largely linear, *logistic regression* (same linear core, sigmoid output) is interpretable, cheap, well-calibrated, and easy to monitor, often the right first production model before reaching for depth.

Answer 1.7*Debugging: weights oscillate and never converge.*

Three distinct causes: (1) **Non-separable data**, the rule cannot converge in principle; confirm by training to a fixed epoch budget and checking whether the misclassification count plateaus above zero, or by trying a soft (logistic) model. (2) **Learning rate too large**, each update overshoots; confirm by logging $\|\mathbf{w}\|$ and the update norm, and by shrinking η to see if oscillation damps. (3) **No proper stopping condition / shuffling artifact**, you keep looping even after separation, or a fixed example order induces a limit cycle; confirm by adding “stop when a full pass has zero errors” and shuffling each epoch.

Answer 1.8*Implement the perceptron rule in NumPy.*

```
perceptron_fit.py
```

```

import numpy as np

def perceptron_fit(X, y, lr=1.0, max_epochs=100):
    """X: (n, d) features; y: (n,) labels in {0,1}. Returns w, b, epochs."""
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for epoch in range(max_epochs):
        errors = 0
        for xi, yi in zip(X, y):
            pred = 1 if (w @ xi + b) >= 0 else 0
            update = lr * (yi - pred)          # 0 when correct
            w += update * xi
            b += update
            errors += int(update != 0)
        if errors == 0:                        # <- the convergence test
            return w, b, epoch + 1
    return w, b, max_epochs

```

The single deciding line is `if errors == 0`: a full pass with no misclassification means a separating hyperplane has been found and training can stop. On non-separable data this is never reached and the loop runs to `max_epochs`.

Answer 1.9

How stacking perceptrons solves XOR at the level of decision regions.

A single hidden unit draws one line, splitting the plane into two half-planes and outputting which side a point is on. With two hidden units you get two independent lines, partitioning the plane into up to four regions. XOR needs the region “inside exactly one of the two half-planes.” Choose $h_1 = \mathbf{1}[x_1 + x_2 - 0.5 \geq 0]$ (OR) and $h_2 = \mathbf{1}[x_1 + x_2 - 1.5 \geq 0]$ (AND); then $\text{XOR} = h_1 \wedge \neg h_2$, which the output unit realizes as $\mathbf{1}[h_1 - h_2 - 0.5 \geq 0]$. The hidden layer *re-represents* the inputs into a space where the classes *are* linearly separable, which is the essence of what depth buys you.

Answer 1.10

Folding bias into the weight vector.

Augment every input with a constant feature: $\tilde{\mathbf{x}} = [\mathbf{x}, 1]$ and $\tilde{\mathbf{w}} = [\mathbf{w}, b]$. Then $\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} = \mathbf{w}^\top \mathbf{x} + b \cdot 1 = \mathbf{w}^\top \mathbf{x} + b$, so the bias becomes an ordinary weight on a constant input, which simplifies the math and the gradient bookkeeping. Frameworks nonetheless keep bias as a separate parameter mainly because of **weight decay**: you almost always want to regularize $\mathbf{w}$ but *not* b (penalizing the bias just shifts the model’s output baseline for no benefit), and a separate parameter makes that exclusion trivial. It also keeps shapes clean for batched matmuls and broadcasting.

Multi-Layer Networks & Forward Pass

Subscribe → aiengineeringinsider.substack.com/subscribe

Stacking neurons into something that can actually learn nonlinear functions.

Chapter 1 ended with a wall: a single perceptron cannot learn XOR, or anything else that isn't linearly separable. This chapter is about the fix, stack neurons into layers, and stack layers into a network, and shows exactly why that fix works, not just that it does.

2.1 Why Depth: The Universal Approximation Theorem

The **universal approximation theorem** (Cybenko, 1989; Hornik et al., 1989) states, informally, that a feedforward network with a single hidden layer of sufficient width, using a non-constant, bounded, continuous activation function, can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary precision.

★ Key Insight

The universal approximation theorem is a statement about what's *representable*, not what's *learnable*. It guarantees a network of the right shape exists; it says nothing about whether gradient descent will actually find it, how much data you'd need, or how wide "sufficient width" really has to be. In practice, that width can be exponentially large for a single hidden layer, which is the entire practical argument for depth over width: deeper networks can represent the same functions with dramatically fewer total parameters.

2.2 Layers, Weights, and Biases as Matrices

A single neuron computes $z = \mathbf{w}^\top \mathbf{x} + b$. A **layer** is just many neurons computing this in parallel, each with its own weight vector, sharing the same input. Stack those weight vectors as rows of a matrix W , and the entire layer's computation becomes a single matrix-vector product:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

For a layer with n inputs and m outputs (neurons), $W \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, and $\mathbf{z} \in \mathbb{R}^m$.

Hidden Layer

Any layer in a network that is neither the input layer nor the output layer. Its outputs are not directly observed; they're an internal, learned representation that the network constructs to make the final task easier.

2.3 The Forward Pass

The **forward pass** is the sequence of computations that turns an input into a prediction. For a network with L layers, each layer applies an affine transformation followed by a nonlinear activation function f :

$$\mathbf{a}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$$

for $l = 1, \dots, L$, where $\mathbf{a}^{(L)}$ is the network's final output. Each $\mathbf{a}^{(l)}$ is called an **activation**, and it's exactly the nonlinearity f applied between layers that breaks the “single straight line” limitation from Chapter 1.

forward_pass.py

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

def forward_pass(x, W1, b1, W2, b2):
    z1 = W1 @ x + b1
    a1 = relu(z1)
    z2 = W2 @ a1 + b2
    # No activation on the final layer here; raw score (e.g. for regression)
    return z2

# A tiny 2-input, 3-hidden-unit, 1-output network
W1 = np.array([[1.0, -1.0], [0.5, 0.5], [-1.0, 1.0]])
b1 = np.array([0.0, 0.0, 0.0])
W2 = np.array([[1.0, 1.0, 1.0]])
b2 = np.array([0.0])

x = np.array([1.0, 0.0])
print(forward_pass(x, W1, b1, W2, b2))
```

Output

[1.5]

2.3.1 Solving XOR with Two Layers

It's worth seeing the resolution to Chapter 1's cliffhanger explicitly. A two-layer network with two hidden units can solve XOR: each hidden unit learns one of the two lines needed to separate the classes, and the output layer combines them.

xor_solution.py

```
import numpy as np

def step(z):
    return (z >= 0).astype(float)

# Hand-picked weights (not learned here, just to show representability)
W1 = np.array([[1.0, 1.0], [1.0, 1.0]])
b1 = np.array([-0.5, -1.5])
W2 = np.array([[1.0, -1.0]])
b2 = np.array([-0.5])

def xor_net(x):
    a1 = step(W1 @ x + b1)
    return step(W2 @ a1 + b2)

for x1 in [0, 1]:
    for x2 in [0, 1]:
        x = np.array([x1, x2])
        print(x1, x2, "->", int(xor_net(x)[0]))
```

Output

```
0 0 -> 0
0 1 -> 1
1 0 -> 1
1 1 -> 0
```

✓ Best Practice

Don't just read this example, run it, then change the hand-picked weights and watch it break. Understanding *why* a particular weight setting solves XOR (what line each hidden unit draws, and how the output layer combines the two half-plane decisions) is worth more than memorizing that a two-layer network “can” solve it.

2.4 Architecture Diagram

Figure 2.1 draws the forward pass of a small multi-layer perceptron as the layered, fully-connected graph frameworks actually build. Inputs enter on the left; each arrow carries a weight; each hidden and output node applies the affine map then the activation f . The vectorized form $\mathbf{a}^{(l)} = f(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$ is just this picture written as matrices.

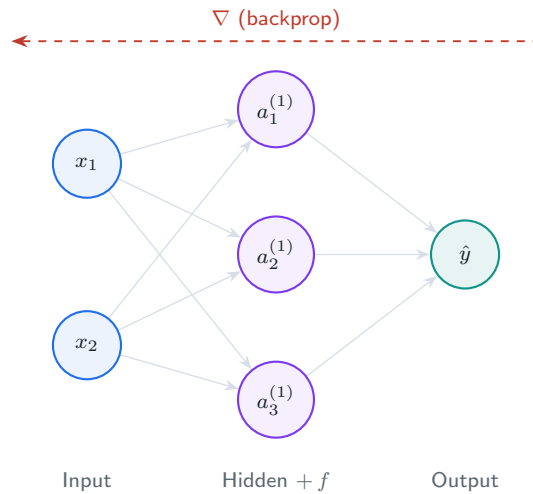


Figure 2.1: Forward pass through a $2 \rightarrow 3 \rightarrow 1$ MLP ($W^{(1)} \in \mathbb{R}^{3 \times 2}$, $W^{(2)} \in \mathbb{R}^{1 \times 3}$). Each layer is a matrix multiply plus bias followed by a nonlinearity; the dashed arrow marks the direction gradients flow during backpropagation (chapter 3).

Figure 2.2 is not a schematic, it is the actual decision boundary a $2 \rightarrow 64 \rightarrow 32 \rightarrow 3$ MLP learns on the three-class “spiral” dataset, produced by the runnable code in [code/gen_figures.py](#). A linear model scores about 33% here; the network reaches 99.3% by bending its boundary into the data’s spiral structure, the visual payoff of stacking nonlinear layers.

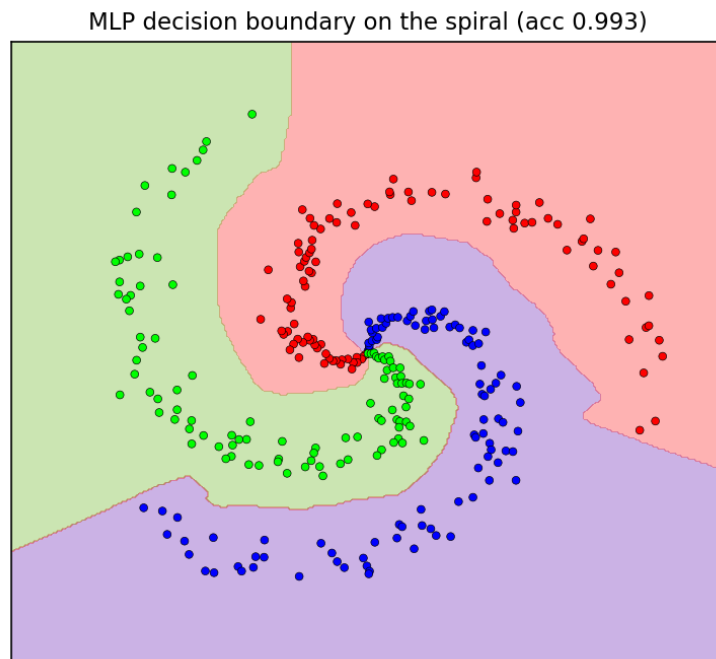


Figure 2.2: Real output: the nonlinear decision regions a trained MLP carves on the three-class spiral (99.3% accuracy). The smooth, curved class boundaries are exactly what a single linear layer (chapter 1) cannot represent.

2.5 Width vs. Depth

Given a fixed parameter budget, should you add more neurons per layer (width) or more layers (depth)? There’s no universal answer, but some general patterns hold in practice:

- **Depth** tends to build hierarchical representations: early layers learn simple features, later layers compose them into increasingly abstract ones (this is most visible in CNNs, Chapter 15).
- **Width** adds raw representational capacity at a given layer but doesn’t on its own encourage hierarchical composition.
- Very deep networks without countermeasures (Chapters 7-10) are harder to train: gradients can vanish or explode as they propagate backward through many layers.
- In practice, architecture search and empirical results for your specific task matter more than either extreme; most production architectures are a deliberate, tuned mix of both.

Table 2.1: Width versus depth at a fixed parameter budget: what each buys and what each costs.

Dimension	Add Width (neurons/layer)	Add Depth (more layers)
Representational gain	Raw capacity at one level	Hierarchical, compositional features
Parameter efficiency	Can need exponentially many units	Often exponentially fewer for same function
Trainability	Easy; gradients stay short	Vanishing/exploding gradients without care
Required tooling	Little	Residuals, normalization, careful init (chapters 8 to 10)
Typical use	Wide heads, embeddings	Backbones (CNNs, transformers)

2.6 Computational Graphs

Frameworks like PyTorch and JAX don’t think about a network as a sequence of equations, they think about it as a **computational graph**: a directed acyclic graph (DAG) where nodes are operations (matrix multiply, add, ReLU) and edges are the tensors flowing between them. The forward pass is a traversal of this graph from inputs to outputs.

Computational Graph

A directed acyclic graph representation of a sequence of mathematical operations, where each node is an operation and each edge carries a tensor (the output of one operation, the input to the next). Building the network this way, rather than as opaque code, is what makes automatic differentiation possible: the graph can be traversed backward to compute gradients, which is exactly the subject of Chapter 3.

This representation matters practically, too: it’s the reason a shape mismatch three layers deep produces an error message pointing at the specific operation node that failed, not a vague failure at the end of the network.

From the Field

A genuinely common first-week bug: defining a layer as `nn.Linear(128, 64)` when the actual input has 256 features, then staring at a cryptic matrix-multiplication shape error for twenty minutes. The fix is almost always faster than the panic suggests: print `x.shape` immediately before the line that fails, and work backward from there. Treat shape-checking as a default debugging reflex, not a last resort.

2.7 Chapter Summary

- The universal approximation theorem guarantees a sufficiently wide single-hidden-layer network can represent any continuous function, but says nothing about learnability or practical width.
- A layer's forward computation is a matrix-vector product followed by a nonlinear activation: $\mathbf{a}^{(l)} = f(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$.
- Two layers are enough to solve XOR: each hidden unit learns one linear boundary, and the output layer combines them nonlinearly.
- Depth tends to build hierarchical representations; width adds raw capacity at a given layer. Most real architectures tune both.
- Frameworks represent networks as computational graphs (DAGs), which is what makes automatic differentiation, and useful shape-mismatch error messages, possible.

2.8 Chapter Evaluation

Ten questions covering representational theory, the forward-pass mechanics, and the practical failure modes of depth. Hints first; answers in section 2.9.

Exercise 2.1

Why do we need a nonlinear activation between layers? Prove it.

Hint. Compose two affine maps and simplify. What kind of function do you get?

Exercise 2.2

The universal approximation theorem says one hidden layer suffices. Why, then, does anyone use deep networks?

Hint. Distinguish “representable” from “learnable,” and think about how width scales.

Exercise 2.3

Write the forward pass of an L -layer MLP in vectorized form and give the shape of every matrix and vector for layer l .

Hint. $\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$. What are the row/column counts of $W^{(l)}$?

Exercise 2.4

Explain, at the level of decision regions, how exactly two hidden units solve XOR.

Hint. Each unit is one line. The output unit combines two half-planes with which Boolean op?

Exercise 2.5

(Production) A teammate adds 8 more `nn.Linear` layers “to increase capacity” but forgets the ReLUs between them. The model trains and loss drops. What’s wrong and how would you catch it in review?

Hint. The loss dropping doesn’t mean capacity increased. What is this 10-layer stack equivalent to?

Exercise 2.6

(Debugging) `nn.Linear(128, 64)` throws a shape error three layers deep. Give your exact debugging procedure.

Hint. The computational graph points at the failing node. What’s the fastest thing to print?

Exercise 2.7

Why is the computational-graph (DAG) view of a network more than a pedagogical nicety? Name two concrete things it enables.

Hint. One is gradients; one is error messages / shape inference.

Exercise 2.8

Given a fixed parameter budget, state two regimes where you’d prefer width and two where you’d prefer depth.

Hint. Embeddings and output heads vs. feature backbones.

Exercise 2.9

Implement a generic L -layer forward pass in NumPy that accepts a list of (W, b) pairs and an activation, with no activation on the final layer.

Hint. Loop over layers; apply f on all but the last.

Exercise 2.10

(Technical) Activations $\mathbf{a}^{(l)}$ are called a “learned representation.” What does that mean operationally, and how would you inspect it?

Hint. Think of the hidden layer as remapping inputs into a space where the task is easier; you can extract and probe those vectors.

2.9 Answers

Answer 2.1

Why a nonlinearity is required, with proof.

A linear layer computes $W\mathbf{x} + \mathbf{b}$. Stack two with nothing between: $W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$, which is again affine, the product of matrices is a matrix. By induction, any number of stacked linear layers collapses to a single equivalent affine map, so depth adds zero representational power. A nonlinear f between layers breaks the collapse, giving the network strictly more expressive power than any single layer.

Answer 2.2

If one hidden layer is universal, why go deep?

The theorem is about *representability*, not *learnability*, and it gives no width bound. For many functions a single hidden layer needs exponentially many units, whereas a deep network represents the same function with far fewer parameters by composing features hierarchically. Depth also tends to produce representations that gradient descent actually finds and that generalize, the theorem promises existence of weights, not that SGD will reach them.

Answer 2.3

Vectorized L -layer forward pass with shapes.

With $\mathbf{a}^{(0)} = \mathbf{x}$, for $l = 1 \dots L$: $\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ and $\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$. If layer l maps n_{l-1} inputs to n_l outputs, then $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$, $\mathbf{b}^{(l)}, \mathbf{z}^{(l)}, \mathbf{a}^{(l)} \in \mathbb{R}^{n_l}$. With a batch of B examples, $A^{(l-1)} \in \mathbb{R}^{n_{l-1} \times B}$ and $Z^{(l)} = W^{(l)}A^{(l-1)} + \mathbf{b}^{(l)}$ (bias broadcast over columns).

Answer 2.4

Decision-region account of two-unit XOR.

Pick $h_1 = \mathbf{1}[x_1 + x_2 - 0.5 \geq 0]$ (fires unless both inputs are 0, an OR) and $h_2 = \mathbf{1}[x_1 + x_2 - 1.5 \geq 0]$ (fires only when both are 1, an AND). XOR is “OR but not AND” = $h_1 \wedge \neg h_2$, which the output unit computes as $\mathbf{1}[h_1 - h_2 - 0.5 \geq 0]$. The hidden layer re-represents the four points into an (h_1, h_2) space where the two classes are linearly separable.

Answer 2.5

Production: stacked Linear layers with no activations.

Without nonlinearities the 10-layer stack is mathematically identical to a single linear layer (Answer 2.1), so it has the representational ceiling of logistic/linear regression: loss can still drop toward the best *linear* fit, masking the bug. Catch it in review by (a) checking every **Linear** is followed by an activation, (b) comparing against an actual one-layer linear baseline, parity in val metrics is the tell, and (c) a unit test that asserts the model can fit a nonlinear toy task (e.g. XOR) it should not be able to fit if it’s secretly linear.

Answer 2.6

Debugging a deep shape error.

The DAG points the traceback at the failing op. Procedure: (1) read the error, it names the operation and the two incompatible shapes; (2) print `x.shape` immediately before that line; (3) walk backward layer by layer checking each `Linear(in,out)`'s `in` against the previous `out`; (4) the mismatch is almost always a hardcoded dimension that doesn't match the real input width. Add a one-time `assert x.shape[-1] == expected` at the model entry to fail fast and clearly next time.

Answer 2.7

Why the DAG view matters.

Two concrete enablers: (1) **Automatic differentiation**, because the graph records every operation and its inputs, it can be traversed in reverse to apply the chain rule (chapter 3); without the recorded graph there is nothing to differentiate through. (2) **Shape inference and precise errors**, the framework propagates tensor shapes through nodes and can report exactly which operation received an incompatible shape, instead of a vague end-of-network failure. (It also enables graph optimizations like operator fusion.)

Answer 2.8

When to prefer width vs depth.

Prefer **width** for: (1) embedding/projection layers and wide output heads where you need raw capacity at one level; (2) shallow tabular models where hierarchy buys little. Prefer **depth** for: (1) perceptual data (images, audio, text) where features are naturally compositional; (2) when parameter efficiency matters, depth represents many functions with exponentially fewer weights. Real architectures tune both (see table 2.1).

Answer 2.9

Generic L-layer forward pass in NumPy.

`mlp_forward.py`

```
import numpy as np

def relu(z): return np.maximum(0.0, z)

def mlp_forward(x, params, activation=relu):
    """params: list of (W, b). No activation on the final layer."""
    a = x
    last = len(params) - 1
    for i, (W, b) in enumerate(params):
        z = W @ a + b
        a = z if i != last else activation(z)    # raw scores on output
    return a
```

This is the exact shape of every framework's **forward**: an affine map per layer, a nonlinearity on all but the output, returning logits/scores.

Answer 2.10

What “learned representation” means, and how to inspect it.

Operationally, $\mathbf{a}^{(l)}$ is the network’s re-encoding of the input into coordinates that make the remaining task easier, e.g. moving XOR points into a linearly-separable (h_1, h_2) space. You inspect it by running inputs forward, capturing a hidden layer’s activations (a forward hook in PyTorch), and probing them: visualize with t-SNE/UMAP, train a cheap linear classifier on them (a “linear probe”), or measure nearest-neighbor class purity. Cleanly separable hidden activations indicate the representation has done useful work.

Backpropagation from Scratch

Subscribe → aiengineeringinsider.substack.com/subscribe

The algorithm that makes deep learning possible, derived step by step.

Chapter 2 showed that stacking layers with nonlinear activations gives a network real representational power, enough to solve XOR and, in principle, approximate any continuous function. But representational power is useless without a way to find the right weights. **Backpropagation** (Rumelhart et al., 1986) is that way, and it is, despite its reputation, nothing more than a disciplined application of the chain rule from calculus.

3.1 The Credit Assignment Problem

Training a network means adjusting every weight, in every layer, to reduce the loss. The hard part isn't computing the loss, it's figuring out *how much each individual weight contributed to that loss*. A weight in the first layer of a ten-layer network affects the output only indirectly, through nine layers of subsequent transformation. This is the **credit assignment problem**: distributing blame (or credit) for the final error back to every parameter that helped cause it.

★ Key Insight

Backpropagation doesn't compute anything new, mathematically. Every gradient it produces could, in principle, be computed by hand with enough patience and the chain rule. What backpropagation provides is an *efficient, systematic algorithm* for computing all of those gradients in a single backward pass, reusing intermediate results instead of recomputing them from scratch for every parameter.

3.2 Chain Rule Refresher

If $y = f(u)$ and $u = g(x)$, then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

This generalizes to longer chains. If $y = f(g(h(x)))$:

$$\frac{dy}{dx} = \frac{dy}{du_1} \cdot \frac{du_1}{du_2} \cdot \frac{du_2}{dx}$$

where $u_1 = g(h(x))$ and $u_2 = h(x)$. A neural network's forward pass is exactly this kind of nested composition, layer after layer, so its gradient is exactly this kind of chained product.

3.2.1 A Worked Scalar Example

Let $x = 2$, and consider $f(x) = (3x + 1)^2$. Let $u = 3x + 1$, so $f = u^2$.

$$\begin{aligned}\frac{df}{du} &= 2u, & \frac{du}{dx} &= 3 \\ \frac{df}{dx} &= \frac{df}{du} \cdot \frac{du}{dx} = 2u \cdot 3 = 6u = 6(3x + 1)\end{aligned}$$

At $x = 2$: $u = 7$, so $\frac{df}{dx} = 42$. Verify numerically: $f(2) = 49$, $f(2.001) \approx 49.042$, and $\frac{49.042 - 49}{0.001} \approx 42$. The chain rule and the numerical derivative agree.

✓ Best Practice

Whenever you derive a gradient by hand, especially for the first time on a new function, check it against a numerical estimate: $\frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$ for a small ϵ (e.g., 10^{-5}). This single habit, called **gradient checking**, will catch sign errors and dropped terms before they cost you a debugging session three layers deep in a real network.

3.3 Backprop Through a Single Neuron

Consider a single neuron: $z = wx + b$, $a = \sigma(z)$ (using sigmoid for concreteness), feeding into a loss L . We want $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$. By the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Each factor is a local, easy derivative: $\frac{\partial z}{\partial w} = x$, $\frac{\partial a}{\partial z} = \sigma(z)(1 - \sigma(z))$ (the sigmoid's own derivative, covered fully in Chapter 5), and $\frac{\partial L}{\partial a}$ depends on the specific loss function (Chapter 6). The same pattern gives $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot 1$, since $\frac{\partial z}{\partial b} = 1$.

Local Gradient

The derivative of a single operation's output with respect to its own input, computed without reference to anything upstream or downstream in the network. Backpropagation works by computing local gradients at every node and chaining them together, rather than trying to derive one giant end-to-end expression by hand.

3.4 Backprop Through a Layer (Vectorized)

Real networks operate on vectors, not scalars, so the gradients become matrix and vector quantities. For a layer $\mathbf{z} = W\mathbf{a}_{in} + \mathbf{b}$, given the upstream gradient $\delta = \frac{\partial L}{\partial \mathbf{z}}$ (often called the layer's **error signal**), the gradients with respect to the layer's own parameters are:

$$\frac{\partial L}{\partial W} = \delta \mathbf{a}_{in}^\top, \quad \frac{\partial L}{\partial \mathbf{b}} = \delta, \quad \frac{\partial L}{\partial \mathbf{a}_{in}} = W^\top \delta$$

That last term, $\frac{\partial L}{\partial \mathbf{a}_{in}}$, is exactly the upstream gradient that gets passed to the *previous* layer, becoming its δ . This is the recursive structure that makes backpropagation a single backward pass rather than N independent derivations for N layers: each layer only needs to know how to compute its own local gradients and pass one quantity backward.

manual_backprop.py

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_grad(z):
    s = sigmoid(z)
    return s * (1 - s)

# Single layer: z = Wx + b, a = sigmoid(z), loss = 0.5*(a - target)^2
W = np.array([[0.5, -0.3]])
b = np.array([0.1])
x = np.array([1.0, 2.0])
target = np.array([0.8])

# Forward pass
z = W @ x + b
a = sigmoid(z)
loss = 0.5 * (a - target) ** 2

# Backward pass
dL_da = (a - target)          # dLoss/da
da_dz = sigmoid_grad(z)      # da/dz
delta = dL_da * da_dz        # combined: dLoss/dz

dL_dW = np.outer(delta, x)    # dLoss/dW
dL_db = delta                 # dLoss/db

print("loss:", loss)
print("dL/dW:", dL_dW)
print("dL/db:", dL_db)
```

Output

```
loss: [0.045]
dL/dW: [[-0.075 -0.15 ]]
dL/db: [-0.075]
```

3.5 Backprop Through Multiple Layers

Stack two layers and the recursion continues exactly as promised: the gradient with respect to layer 2's input becomes layer 1's δ . The full algorithm, for a network with L layers, is:

1. Run the forward pass, caching every $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$ along the way (you need them again on the backward pass).
2. Compute $\delta^{(L)} = \frac{\partial L}{\partial \mathbf{a}^{(L)}} \odot f'(\mathbf{z}^{(L)})$ at the output layer.
3. For $l = L - 1, \dots, 1$: compute $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot f'(\mathbf{z}^{(l)})$.
4. At every layer, $\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \mathbf{a}^{(l-1)\top}$ and $\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$.

($\odot$ denotes element-wise multiplication.) Notice the caching requirement in step 1: this is why training a network uses meaningfully more memory than just running inference, every intermediate activation has to stay in memory until the backward pass consumes it.

3.6 Architecture Diagram

Figure 3.1 shows the two passes that define training. The forward pass (top, blue) flows left to right, caching each $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$. The backward pass (bottom, red) flows right to left, threading the error signal δ through the transpose-weight recursion $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot f'(\mathbf{z}^{(l)})$ and reading the cached values to form $\partial L / \partial W^{(l)}$.

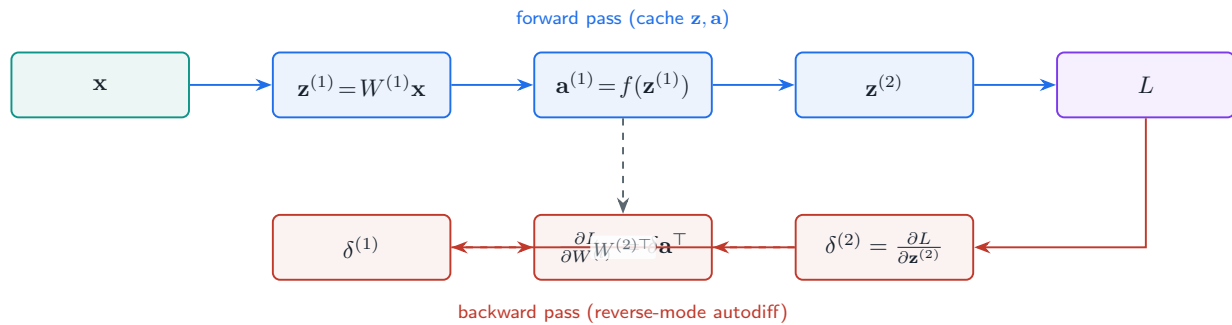


Figure 3.1: Forward and backward passes over a two-layer network. The cache (dashed, center) is why training costs more memory than inference: every activation must survive until the backward pass consumes it.

Table 3.1: Reverse-mode versus forward-mode automatic differentiation, and why deep learning uses reverse mode.

Property	Reverse mode (backprop)	Forward mode
Sweep direction	Outputs $\rightarrow$ inputs	Inputs $\rightarrow$ outputs
Cost per pass	One scalar loss $\rightarrow$ all params	One input $\rightarrow$ all outputs
Efficient when	Many params, few outputs (DL)	Few inputs, many outputs
Memory	High (caches activations)	Low (no cache)
Used by	PyTorch <code>.backward()</code> , JAX <code>grad</code>	JAX <code>jvp</code> , Jacobian columns

3.7 Computational Graphs and Reverse-Mode Autodiff

The manual derivation above generalizes to arbitrary computational graphs (Chapter 2), not just stacks of identical layers. **Reverse-mode automatic differentiation** traverses the graph

backward from the loss, applying the chain rule at each node using only that node’s local gradient and the gradient flowing in from downstream. This is exactly what `.backward()` does in PyTorch and what `grad()` does in JAX, and it’s exactly what you’ll implement yourself in Chapter 4.

Reverse-Mode Automatic Differentiation

An algorithm for computing exact gradients of a scalar output (typically the loss) with respect to every input or parameter in a computational graph, by traversing the graph in reverse topological order and applying the chain rule at each node. It is called “reverse-mode” because it propagates from outputs back to inputs, which is the efficient direction when there are many parameters and one scalar loss, exactly the situation in deep learning.

3.8 Common Derivation Mistakes

! Pitfall / Warning

The single most common hand-derivation mistake is dropping a transpose or getting matrix dimensions backward, $\frac{\partial L}{\partial W} = \delta \mathbf{a}_{in}^\top$ and $\frac{\partial L}{\partial \mathbf{a}_{in}} = W^\top \delta$ use the transpose in different places for a reason: the dimensions have to work out so the result matches the shape of W or $\mathbf{a}_{in}$, respectively. When in doubt, write out the dimensions explicitly ($W \in \mathbb{R}^{m \times n}$, $\delta \in \mathbb{R}^m$, $\mathbf{a}_{in} \in \mathbb{R}^n$) and check that every product is dimensionally valid before trusting the derivation.

The second most common mistake: forgetting that $\delta^{(l)}$ depends on $f'(\mathbf{z}^{(l)})$, the derivative of *that layer’s own* activation function, not the next layer’s. Mixing these up silently produces a network that trains, just incorrectly, which is far harder to debug than a network that crashes outright. This is precisely why vanishing and exploding gradients (introduced next) are so insidious: the symptom (a network that won’t learn) looks identical whether the cause is a mathematical bug or simply many layers of small derivatives multiplying together.

3.9 Vanishing and Exploding Gradients: First Encounter

Look again at the recursive formula for $\delta^{(l)}$: it involves a product of $f'(\mathbf{z})$ terms across every layer from L back to l . If those derivatives are consistently less than 1 (true of sigmoid almost everywhere, where the maximum derivative is just 0.25), the product shrinks exponentially with depth, the **vanishing gradient problem**. Early layers in a deep network can receive a gradient signal so small that they effectively stop learning. The reverse problem, derivatives consistently greater than 1, causes gradients to grow exponentially instead, the **exploding gradient problem**, leading to unstable, divergent training.

! Pitfall / Warning

This isn’t a one-time, historical concern, it’s the underlying reason several entire chapters of this book exist: activation function choice (Chapter 5), careful initialization (Chapter 8), normalization layers (Chapter 9), and architectures like LSTM (Chapter 16) are all, in part, direct responses to vanishing and exploding gradients. If a network refuses to train and the loss is flat or NaN, this family of problems should be among your first suspects.

3.10 Chapter Summary

- Backpropagation is the chain rule, applied systematically and efficiently across a computational graph, nothing more exotic than that.
- Each node only needs its own local gradient; the chain rule combines local gradients into the full end-to-end gradient.
- The backward pass is recursive: each layer's output gradient becomes the previous layer's incoming error signal δ .
- Reverse-mode automatic differentiation is the general algorithm underlying `.backward()` in PyTorch and `grad()` in JAX.
- Vanishing and exploding gradients arise from repeated multiplication of per-layer derivatives across depth, and motivate much of the rest of this book's training-mechanics content.

3.11 Chapter Evaluation

Ten questions on the chain rule, the vectorized recursion, memory cost, and the numerical failure modes backprop is prone to. Hints first; answers in section 3.12.

Exercise 3.1

Derive $\partial L / \partial z$ for cross-entropy with a sigmoid output and explain why the result is so clean.

Hint. Multiply $\partial L / \partial a$ by the sigmoid derivative $a(1 - a)$ and watch terms cancel.

Exercise 3.2

What is the precise difference between backpropagation and gradient descent?

Hint. One computes gradients; the other consumes them. They are separable.

Exercise 3.3

For a layer $\mathbf{z} = W\mathbf{a}_{in} + \mathbf{b}$, derive the three gradients and explain why the transpose appears where it does.

Hint. Match shapes: $\partial L / \partial W$ must be shaped like W ; $\partial L / \partial \mathbf{a}_{in}$ like $\mathbf{a}_{in}$.

Exercise 3.4

Why does training require caching forward activations, and what does that imply for memory at scale?

Hint. Step 4 of the algorithm uses $\mathbf{a}^{(l-1)}$. When is it consumed?

Exercise 3.5

Explain vanishing and exploding gradients from the recursion, and name three countermeasures and the chapters that cover them.

Hint. $\delta^{(l)}$ multiplies $f'(\mathbf{z})$ across every layer. What if each factor is < 1 ? > 1 ?

Exercise 3.6

(Debugging) Your loss goes to NaN after a few hundred steps. Walk through how you'd localize whether it's exploding gradients, a bad learning rate, or a numerical issue in the loss.

Hint. Log gradient norms; clip; check for $\log(0)$ in the loss.

Exercise 3.7

Implement gradient checking for an arbitrary scalar function and state the tolerance you'd accept.

Hint. Central difference $(f(x + \epsilon) - f(x - \epsilon))/2\epsilon$; compare relative error.

Exercise 3.8

Why is reverse mode (not forward mode) the right autodiff direction for deep learning? When would forward mode win?

Hint. Count the cost in terms of number of parameters versus number of outputs.

Exercise 3.9

(Debugging) Your hand-derived gradient trains a network that learns, but slightly worse than the framework. The most likely class of bug?

Hint. A network that trains-but-wrongly usually has a subtle, silent error, e.g. the wrong layer's f' .

Exercise 3.10

(Technical) Gradient checkpointing trades compute for memory. Explain the mechanism in terms of the activation cache.

Hint. If you don't store an activation, how do you get it back during the backward pass?

3.12 Answers

Answer 3.1

Gradient of cross-entropy w.r.t. the logits.

With $a = \sigma(z)$ and $L = -[y \log a + (1 - y) \log(1 - a)]$: $\frac{\partial L}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$ and $\frac{\partial a}{\partial z} = a(1 - a)$. Their product is $(-\frac{y}{a} + \frac{1-y}{1-a})a(1 - a) = -y(1 - a) + (1 - y)a = a - y$. So $\frac{\partial L}{\partial z} = a - y$, prediction minus label. It is clean because the sigmoid's $a(1 - a)$ exactly cancels the $1/a$ and $1/(1 - a)$ from the log loss; this is why sigmoid/softmax is paired with cross-entropy and why you should feed *logits* (not probabilities) into the loss for numerical stability.

Answer 3.2

Backprop vs gradient descent.

Backpropagation *computes* $\partial L / \partial \theta$ for every parameter by reverse-mode autodiff. Gradient descent *uses* those gradients to update: $\theta \leftarrow \theta - \eta \partial L / \partial \theta$. Backprop tells you the ascent

direction; gradient descent is the policy of stepping opposite to it. They are separable, you can pair backprop's gradients with momentum, Adam, or AdamW (chapter 7) instead of vanilla descent.

Answer 3.3

Layer gradients and the transpose.

$\frac{\partial L}{\partial \mathbf{W}} = \delta \mathbf{a}_{in}^\top$, $\frac{\partial L}{\partial \mathbf{b}} = \delta$, $\frac{\partial L}{\partial \mathbf{a}_{in}} = \mathbf{W}^\top \delta$. With $\mathbf{W} \in \mathbb{R}^{m \times n}$, $\delta \in \mathbb{R}^m$, $\mathbf{a}_{in} \in \mathbb{R}^n$: the outer product $\delta \mathbf{a}_{in}^\top$ is $m \times n$ (matches $\mathbf{W}$); $\mathbf{W}^\top \delta$ is $n \times 1$ (matches $\mathbf{a}_{in}$). The transpose appears wherever needed to make the product dimensionally valid and shaped like the quantity being differentiated, the single fastest derivation sanity check.

Answer 3.4

Why caching, and the memory implication.

$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \mathbf{a}^{(l-1)\top}$ needs the *forward* activation $\mathbf{a}^{(l-1)}$, which isn't available until the backward pass reaches layer l . So every activation must be retained from the forward pass until consumed, training memory scales with depth $\times$ batch $\times$ activation size, far above inference. This is why batch size is memory-bound and why gradient checkpointing (Answer 3.10) exists.

Answer 3.5

Vanishing/exploding gradients and countermeasures.

$\delta^{(l)}$ is a product of $f'(\mathbf{z})$ and weight terms across layers $L \rightarrow l$. If those factors are consistently < 1 (sigmoid's max is 0.25) the product shrinks exponentially with depth, early layers stop learning (vanishing). If consistently > 1 , it grows exponentially (exploding, often NaN). Countermeasures: non-saturating activations like ReLU/GELU (chapter 5), variance-preserving initialization (chapter 8), normalization layers (chapter 9), residual connections, gradient clipping, and gated recurrence/LSTM (chapter 16).

Answer 3.6

Debugging a loss that goes NaN.

Localize in order: (1) **Loss math**, check for $\log(0)$ / $0/0$, use the logits-based loss (`BCEWithLogits`, `CrossEntropyLoss`) rather than logging a probability. (2) **Exploding gradients**, log the global grad norm each step; if it spikes before the NaN, add `clip_grad_norm` and/or lower the LR. (3) **Learning rate**, halve it; if NaN disappears it was a step-size/stability issue. (4) Check inputs/targets for NaN/Inf and that you're not dividing by a near-zero std in normalization.

Answer 3.7

Gradient checking.

`grad_check.py`

```

import numpy as np

def grad_check(f, x, analytic_grad, eps=1e-5, tol=1e-6):
    """f: scalar fn of vector x. analytic_grad: your computed gradient."""
    num = np.zeros_like(x, dtype=float)
    for i in range(x.size):
        xp, xm = x.copy(), x.copy()
        xp[i] += eps; xm[i] -= eps
        num[i] = (f(xp) - f(xm)) / (2 * eps)      # central difference
    rel = np.abs(num - analytic_grad) / (np.abs(num) + np.abs(analytic_grad) +
    ↪ 1e-12)
    return rel.max(), rel.max() < tol              # accept if max rel err < tol

```

Use the *central* difference (error $O(\epsilon^2)$, not $O(\epsilon)$), and judge by *relative* error; $< 10^{-6}$ is solid for double precision, relax to $\sim 10^{-3}$ for float32.

Answer 3.8

Why reverse mode for deep learning.

One reverse pass yields the gradient of a single scalar (the loss) w.r.t. *all* parameters at cost $\approx$ one forward pass, ideal when params are many and outputs are one. Forward mode costs one pass *per input direction*, so it wins only when inputs are few and outputs many (e.g. a Jacobian of a function $\mathbb{R}^k \rightarrow \mathbb{R}^m$ with $k \ll m$, or Jacobian-vector products). Deep nets have millions of params and a scalar loss, reverse mode is asymptotically cheaper.

Answer 3.9

Debugging trains-but-worse.

A silent gradient bug, almost always using the wrong layer's activation derivative (applying $f'(\mathbf{z}^{(l+1)})$ where $f'(\mathbf{z}^{(l)})$ is required), a dropped transpose, or accumulating with $=$ instead of $+=$ on a shared parameter. The network still descends *some* surrogate, so loss falls, just along the wrong direction. Confirm with gradient checking (Answer 3.7) on a tiny instance against numerical gradients.

Answer 3.10

Gradient checkpointing.

Normally every forward activation is cached for the backward pass. Checkpointing stores only a sparse subset (the “checkpoints”) and *recomputes* the dropped activations on the fly during the backward pass by re-running the forward segment from the nearest checkpoint. It trades extra compute (one partial re-forward) for a large drop in peak memory ($O(\sqrt{L})$ instead of $O(L)$ for an L -layer net), which is how very deep/large models fit on a given GPU.

Build Your Own Mini Framework

Subscribe → aiengineeringinsider.substack.com/subscribe

A micrograd-style autograd engine, to prove you actually understand Chapter 3.

There's a specific kind of understanding that only comes from implementation. You can read Chapter 3's derivations, nod along, and still not really *know* backpropagation the way you will after building it yourself, end to end, in maybe 100 lines of Python. This chapter walks through exactly that exercise, in the tradition popularized by Andrej Karpathy's `micrograd`.

4.1 Why Build a Toy Framework

★ Key Insight

The goal here is not to produce something useful, PyTorch and JAX already exist, and Chapters 12 and 13 cover them in depth. The goal is to remove every shred of “magic” from automatic differentiation by implementing it yourself, so that when you later call `loss.backward()` in PyTorch, you have a precise mental model of what's actually happening underneath, not a black box you trust on faith.

4.2 Designing a Value Class

The core idea: wrap every number in an object that remembers (a) its current value, (b) its gradient (initialized to zero), and (c) how it was produced, which other `Value` objects, and which operation, created it. That third piece of information is what lets us walk the computational graph backward later.

value_class.py

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None  # how to propagate gradient backward
        self._prev = set(_children)    # parent nodes in the graph
        self._op = _op                 # which operation produced this value

    def __repr__(self):
        return f"Value(data={self.data}, grad={self.grad})"
```

Every arithmetic operation we implement will do two things: compute the forward result as a new `Value`, and attach a `_backward` function that knows how to push gradient from that result back onto its inputs.

4.3 Implementing Forward Operations

Start with addition and multiplication, the building blocks of $z = wx + b$.

`ops_add_mul.py`

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')

        def _backward():
            self.grad += out.grad
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')

        def _backward():
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def __repr__(self):
        return f"Value(data={self.data}, grad={self.grad})"
```

✓ Best Practice

Notice the `+=` in every `_backward` function, not `=`. If a single `Value` feeds into more than one downstream operation (a very common situation, e.g. a bias term used in multiple places), its gradient must *accumulate* contributions from every path. Using plain assignment here is one of the most common bugs when implementing autograd from scratch, and it fails silently: the forward pass still works, the gradients are just quietly wrong.

Next, add power (for things like squared error) and a nonlinearity. We'll implement `tanh` here as a concrete, smooth, differentiable example.

ops_pow_tanh.py

```

import math

class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    # ... __add__ and __mul__ as above ...

    def __pow__(self, other):
        assert isinstance(other, (int, float))
        out = Value(self.data ** other, (self,), f'**{other}')

        def _backward():
            self.grad += (other * self.data ** (other - 1)) * out.grad
            out._backward = _backward
        return out

    def tanh(self):
        t = math.tanh(self.data)
        out = Value(t, (self,), 'tanh')

        def _backward():
            self.grad += (1 - t ** 2) * out.grad
            out._backward = _backward
        return out

```

Each `_backward` function above is exactly the local gradient from Chapter 3, written in code instead of on paper: $\frac{d}{dx}x^n = nx^{n-1}$, and $\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$.

Finally, add a few convenience operators so a `Value` behaves like an ordinary number, these are what let the training loop write `yp - yt` and `sum(...)` directly. Without them the loop below raises a `TypeError`.

value_ops_extra.py

```

def __neg__(self):                # -self
    return self * -1

def __sub__(self, other):         # self - other
    return self + (-other)

def __radd__(self, other):       # other + self (so sum() works)
    return self + other

```

4.4 The Backward Pass: Topological Sort

Here's the part that ties everything together. To correctly accumulate gradients, we must process nodes in *reverse topological order*, every node's gradient must be fully accumulated (from all its consumers) *before* we call its own `_backward` to push gradient further upstream. Getting this order wrong silently produces incorrect gradients on any graph with branching.

`backward_method.py`

```
def backward(self):
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)
    build_topo(self)

    self.grad = 1.0 # seed: dL/dL = 1
    for v in reversed(topo):
        v._backward()
```

`Value.backward = backward`

From the Field

The most common bug students hit when building this for the first time isn't in the math, it's exactly this ordering. If you call `_backward()` in the wrong order (say, forward topological order instead of reversed), some nodes will push gradient before they've received all of their own incoming gradient, and the result is wrong in a way that often *looks* plausible. If your hand-computed gradients don't match this engine's output, check the topological sort before you suspect the math.

4.5 Architecture Diagram

Figure 4.1 shows the computational graph our engine builds for the expression $c = a \cdot b + a^2$ (the one we gradient-check at the end of the chapter). Solid arrows are the forward pass that records the graph; dashed red arrows are the reverse-topological backward pass that accumulates `.grad` on each node. Note that a feeds two paths, exactly why gradients must `+=` rather than overwrite.

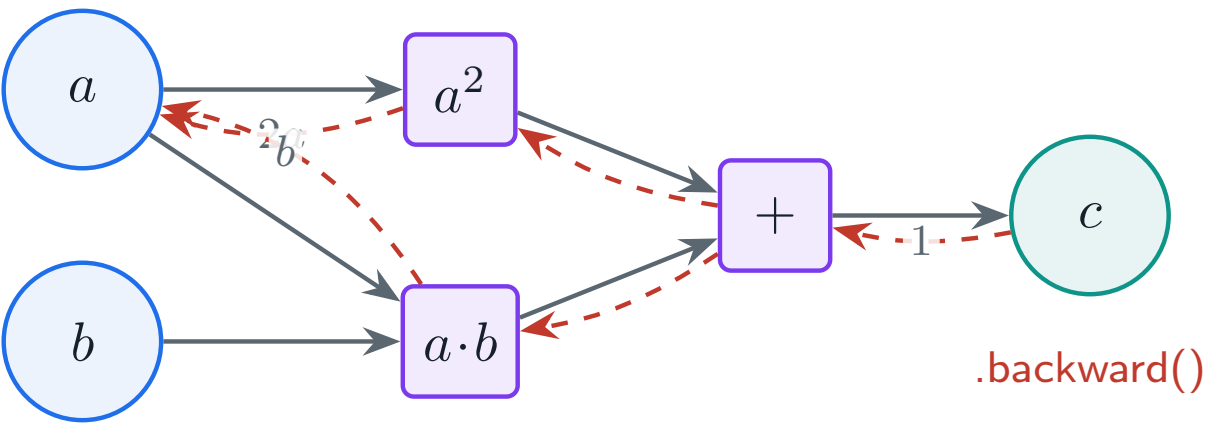


Figure 4.1: The autograd graph for $c = a \cdot b + a^2$. Each node stores `data`, `grad`, its parents, and a `_backward` closure. The node `a` has two consumers, so its gradient is the sum of both incoming contributions.

Table 4.1: Our scalar engine versus production tensor frameworks: same principles, very different engineering.

Aspect	This chapter’s engine	PyTorch / JAX
Granularity	Scalar Value	N-dim tensors (vectorized)
Backward	Python closures, recursion	Fused C++/CUDA kernels
Graph	Built every forward (define-by-run)	Eager (PyTorch) / traced & compiled (JAX)
Hardware	CPU, one number at a time	GPU/TPU, massively parallel
Purpose	Understanding	Production performance

4.6 Building a Tiny MLP

With `Value` as the foundation, a neuron, a layer, and a full MLP are thin wrappers around composing `Value` operations.

```
mini_mlp.py

import random

class Neuron:
    def __init__(self, nin):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(0.0)

    def __call__(self, x):
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        return act.tanh()

    def parameters(self):
        return self.w + [self.b]

class Layer:
    def __init__(self, nin, nout):
```

```

        self.neurons = [Neuron(nin) for _ in range(nout)]

    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs[0] if len(outs) == 1 else outs

    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]

class MLP:
    def __init__(self, nin, nouts):
        sizes = [nin] + nouts
        self.layers = [Layer(sizes[i], sizes[i+1]) for i in range(len(nouts))]

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]

```

4.7 Training It End to End

A complete training loop: forward pass, compute loss, zero gradients, backward pass, update weights. This is the exact same loop you'll write in PyTorch in Chapter 12, just with your own engine underneath instead of PyTorch's.

`train_loop.py`

```

import random
random.seed(42)    # reproducible run (the output below is from this seed)

# Toy dataset: 4 examples, 3 input features, binary-ish targets
xs = [
    [2.0, 3.0, -1.0],
    [3.0, -1.0, 0.5],
    [0.5, 1.0, 1.0],
    [1.0, 1.0, -1.0],
]
ys = [1.0, -1.0, -1.0, 1.0]

model = MLP(3, [4, 4, 1])

for step in range(50):
    # Forward pass
    ypred = [model(x) for x in xs]
    loss = sum((yp - yt) ** 2 for yp, yt in zip(ypred, ys))

    # Zero gradients

```

```

for p in model.parameters():
    p.grad = 0.0

# Backward pass
loss.backward()

# Update (plain SGD, Chapter 7 covers better optimizers)
for p in model.parameters():
    p.data += -0.05 * p.grad

if step % 10 == 0:
    print(f"step {step:2d}  loss {loss.data:.4f}")

```

Output

```

step 0  loss 5.2788
step 10  loss 0.3966
step 20  loss 0.1106
step 30  loss 0.0577
step 40  loss 0.0377

```

★ Key Insight

Watch the loss column. It should decrease, monotonically or close to it, exactly the same signature you'll learn to read in real PyTorch training runs in Chapter 11 and Chapter 14. If you implement this yourself and the loss does *not* decrease, that's valuable: it means you have a real bug to chase down with a 100-line, fully-inspectable codebase, far easier debugging practice than chasing the same kind of bug inside a production PyTorch model.

4.8 Gradient Checking Your Engine

Before trusting any of this, verify it against numerical differentiation, exactly the technique introduced in Chapter 3.

gradient_check.py

```

def numerical_grad(f, x, eps=1e-6):
    return (f(x + eps) - f(x - eps)) / (2 * eps)

# Compare engine gradient to numerical gradient for a small expression
a = Value(2.0)
b = Value(-3.0)
c = a * b + a ** 2
c.backward()

def f(a_val):
    return a_val * (-3.0) + a_val ** 2

```

```
print("engine dC/da:    ", a.grad)
print("numerical dC/da: ", numerical_grad(f, 2.0))
```

Output

```
engine dC/da:    1.0
numerical dC/da: 1.000000000139778
```

✓ Best Practice

Make gradient checking a permanent part of your workflow, not a one-time exercise. Any time you implement a new operation in a custom autograd system, or suspect a gradient bug in any framework, write a five-line numerical check like the one above before you spend an hour staring at training curves.

4.9 Chapter Summary

- A minimal autograd engine wraps scalars in a `Value` class that tracks data, gradient, and the operation that produced it.
- Every operation defines both a forward computation and a local backward function, exactly the local gradients derived by hand in Chapter 3.
- Gradients must accumulate (`+=`), not overwrite, since a value can feed into multiple downstream operations.
- The backward pass requires reverse topological order; getting this wrong is the most common bug when implementing autograd from scratch.
- A full MLP, and a full training loop, are thin compositions on top of this engine, the exact same shape you'll see in PyTorch in Chapter 12.

4.10 Chapter Evaluation

Ten questions on autograd internals, the classic from-scratch bugs, and how the toy engine maps to real frameworks. Hints first; answers in section 4.11.

Exercise 4.1

Walk through how autograd works under the hood, from forward graph construction to `.backward()`.

Hint. Record nodes on the forward pass; topologically sort; seed $dL/dL = 1$; chain local gradients in reverse.